



Model Families

Noam Barda
MD, PhD

&

Noa Dagan
MD, PhD, MPH



Motivation

- Your design is set, your data is ready, and you want to fit a model
- But which model?
- And with which hyper-parameters?
- For the purposes of this course, we will assume that you know
 - The important model families
 - How to fit a model
 - Basic learning theory principles such as the bias/variance trade-off
- Instead, we'll focus on models that are widely used in healthcare and specifically on survival analysis, which is not usually covered in standard DS courses

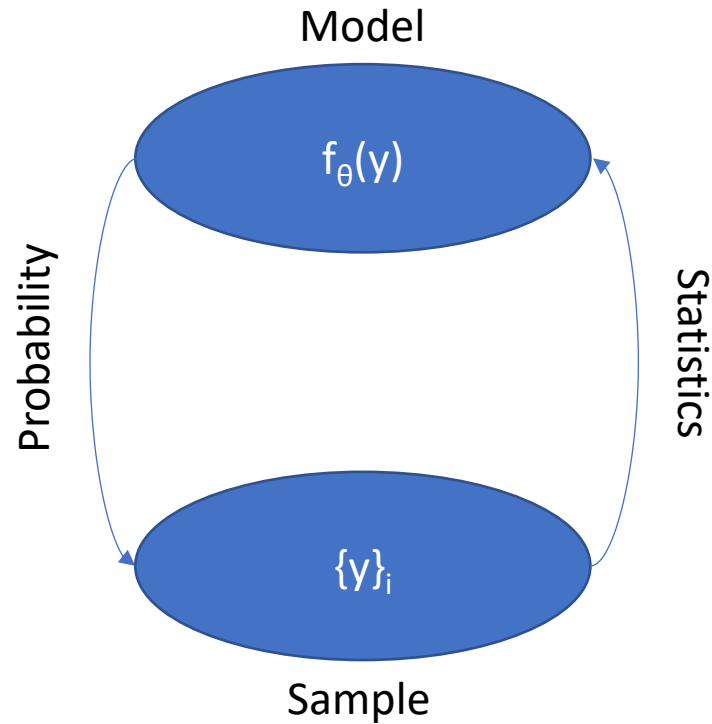
Popular Models in Clinical Prediction

(Focusing on structured data)

- Regressions
 - Logistic regression (sometimes regularized)
 - Poisson regression
 - Cox regression
- Tree-based methods
 - Random forests
 - Gradient boosted decision trees (xgboost, etc.)
 - Random survival forests

A Note about Discussing Models

- We will mostly be looking at things from a statistical viewpoint



A Note about Discussing Models (2)

- So, for a given model family (say linear regression), we will describe the data generating process it assumes about the world (the “model”) but will not discuss the estimation process at length (or at all). Just assume we have a way to estimate the parameters
- Computer science DS courses sometimes put more emphasis on the algorithmic parts, maybe because in a lot of the models there are no clear assumptions about the DGP
- Regardless, it would be useful for you to feel comfortable with the “I specify a model, and assume that it can be estimated somehow” approach

A Note about Discussing Models (3)

- Some analytic traditions (statistics, epidemiology) favor modeling the data generating process faithfully
- Some analytic traditions (econometrics) put less emphasis on that, instead willing to go for any estimator that “gets the job done”
- We will see examples of both

Towards Generalized Linear Models (GLMs)

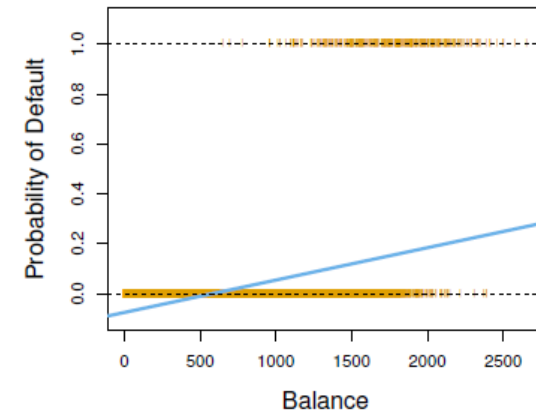
- At the dawn of time, we had linear regression

$$Y \sim \text{Normal}(\mu, \sigma^2),$$

$$\mu = \sum_{i=1}^n \beta_i x_i$$

- This is useful for continuous outcomes, but what about other outcomes (binary? count? time-to-event?)
- And you should know that most outcomes in the biomedical sciences are NOT continuous. You could have a full and gratifying career in medical data science and never once encounter a continuous outcome

But why not Linear Regression?



- Say we have a binary outcome
- We could view the linear predictor as an estimate for $P(Y|X)$
- And economists actually do love using linear regression for causal inference, which they dub a “linear probability model” (LPM)
- But we are discussing prediction, and
 - It doesn’t work when there are more than two levels to the outcome (because then, what ordering do you use?)
 - It can predict outside the $[0,1]$ range, creating meaningless predictions
- (This is an example of the “any good estimator will do” approach)

GLMs

- Enter generalized linear models

$$Y \sim \text{Exponential} - \text{family}(\mu)$$

$$\mu = g^{-1}(\eta)$$

$$\eta = \sum_{i=1}^n \beta_i x_i$$

- Let's break it down to its three elements:
 1. Y is sampled from an exponential family distribution (the exponential family, for those not familiar, covers a lot of useful families, including the Gaussian, Binomial and Poisson distributions)
 2. One parameter of this distribution is a function of the linear predictor
 3. A link function, g, is used to translate the linear predictor (which is defined over $\mathbb{R}$) to the domain of the parameter (e.g., a probability for Bernoulli, a non-negative number for Poisson, etc.)

Logistic Regression

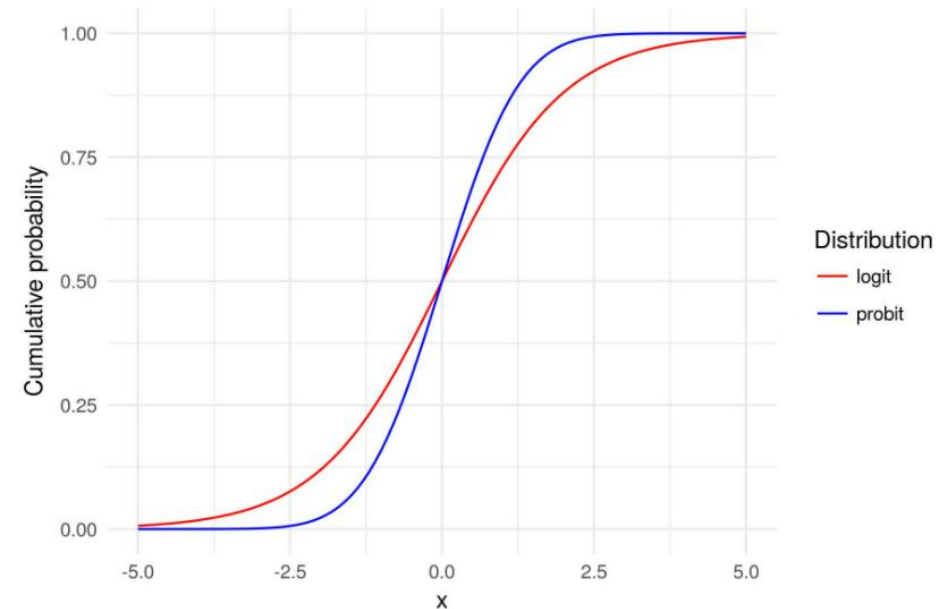
$$Y \sim \text{Bernoulli}(P)$$

$$\log\left(\frac{P}{1-P}\right) = \sum_{i=1}^n \beta_i x_i$$

- The distribution is Bernoulli, and the link function is the logit
- To go back from the linear predictor to the prediction we use the “expit” function: $p(X) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}$
- The model is used for binary outcomes, and is the work-horse of biostatistics

Logistic Regression (2)

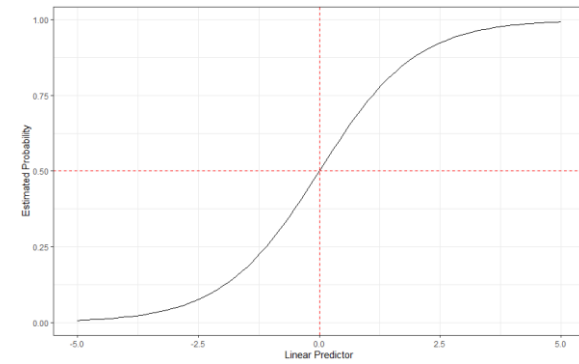
- There is nothing special about the logit function other than that it maps from $[0,1]$ to $\mathbb{R}$
- Other link functions exist and are used, mostly in econometrics
- E.g., Probit regression = same thing, but uses the inverse of the gaussian CDF as the link function



Logistic Regression (3)

- When trying to understand the model or when doing non-prediction tasks, it's very nice to have a useful interpretation of the model parameters
- But similar to most other GLMs, logistic regression doesn't get the privilege of a "unit change in x always translates into a β change in the outcome" (the outcome here being the probability)
- It does translate into a β change in the "linear predictor", but that's not directly useful
- What then?

Logistic Regression (4)



- Imagine two people, A and B, that differ by one unit in x_1

$$\beta_1 = \log\left(\frac{P(A)}{1 - P(A)}\right) - \log\left(\frac{P(B)}{1 - P(B)}\right) = \log\left(\frac{\frac{P(A)}{1 - P(A)}}{\frac{P(B)}{1 - P(B)}}\right) \Rightarrow$$
$$e^{\beta_1} = \frac{\frac{P(A)}{1 - P(A)}}{\frac{P(B)}{1 - P(B)}}$$

- We would say that the exponent of the coefficient is the **odds ratio**
- So, a unit change in the variable multiplies the odds
- But it's still a weird relationship with “edge effects” on the probability scale
- It's also always in interaction, because, e.g., an OR change around 0.5 is a much larger change in probability than an OR change around 0.9

Margins

- That ORs are constant with respect to the other Xs is great
- But ORs are not inherently interpretable
 - A OR=2 “jump” from $P=0.5$ gets us to $P=0.67$ ($\Delta P=0.17$)
 - Another OR=2 “jump” then gets us to $P=0.8$ ($\Delta P=0.13$)
 - And then $\Delta P=0.09$, ...
- Instead, a different modeling tradition chooses to quantify the effect directly on the ΔP scale
 - In practice, we average over the Xs observed in the sample
 - So, a “marginal” effect, which marginalizes out the Xs
- This is very interpretable, but depends on the distribution of the Xs, which are then assumed to be from an interesting sample

Logistic Regression (5)

- Logistic regression is usually estimated using the maximum likelihood estimator.

$$\ell(\theta) = \sum_i y_i \log(P_i) + \sum_i (1 - y_i) \log(1 - P_i)$$

- There is no closed-form solution, so the argmax is found using numerical techniques (gradient ascent, etc.)
- The MLE has very useful statistical properties (specifically, it is CAN - consistently asymptotically normal)

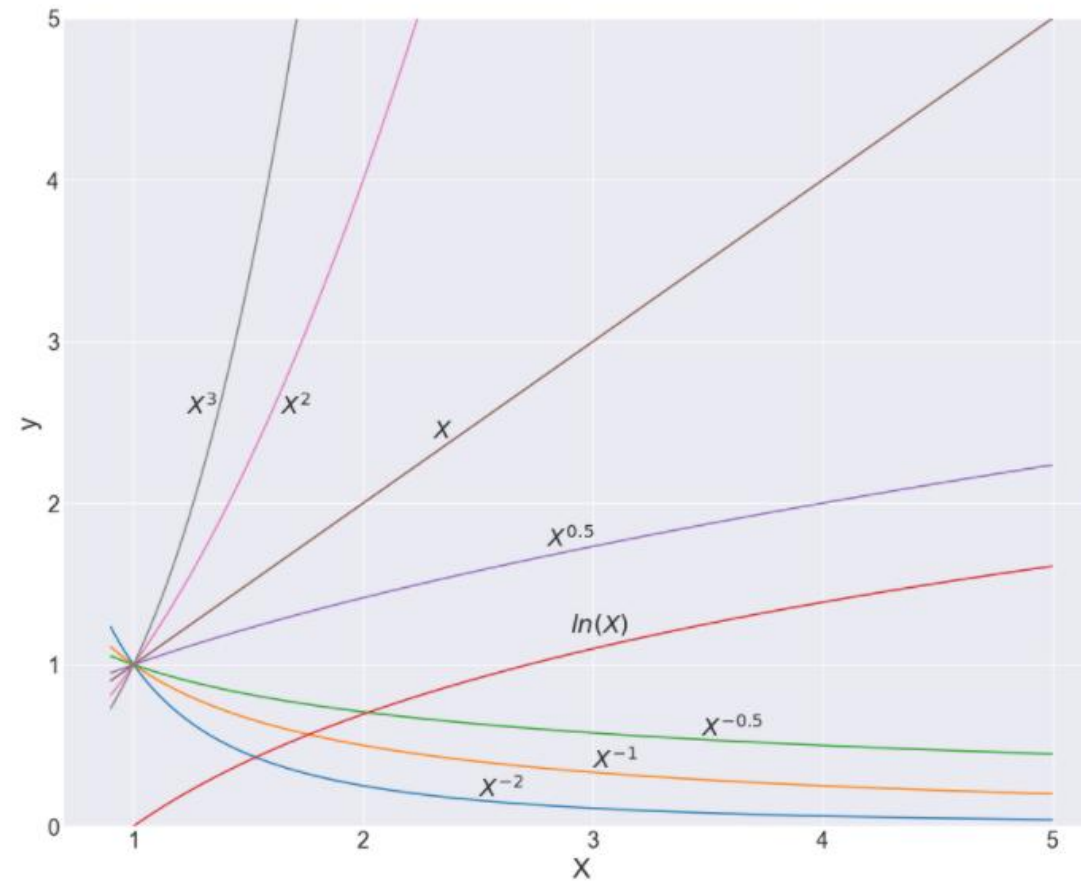
Logistic Regression (6)

- Categorical variables are handled using dummy coding
- Predictions are had by calculating the linear predictor then running expit on that
- We get a probability (as opposed to a 0/1 classification), which allows us to talk about AUC, calibration, decision analysis, etc.

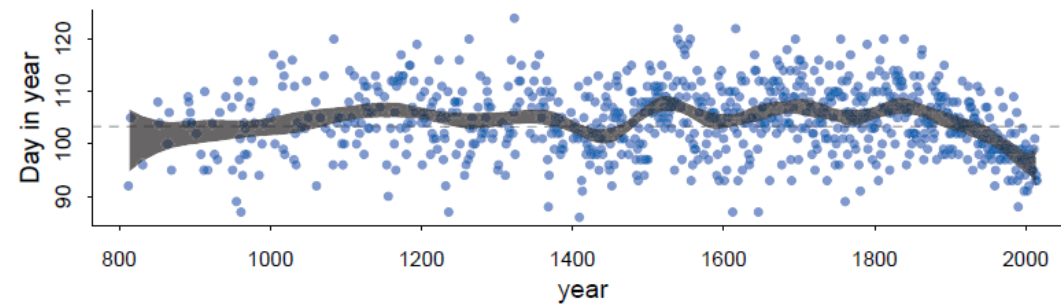
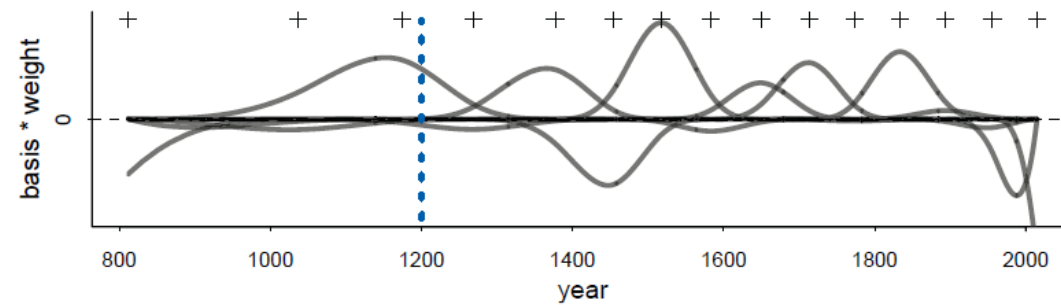
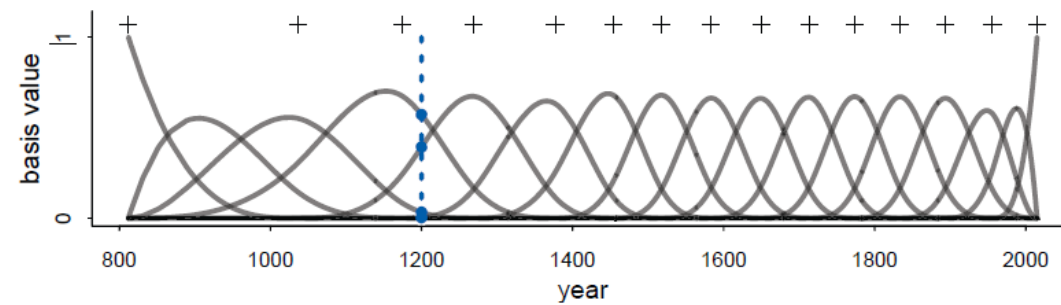
Logistic Regression (7)

- Linear models are linear in the parameters (e.g., it's always $\sum_{i=1}^n \beta_i x_i$ and never $\sum_{i=1}^n \beta_i^2 x_i$)
- But you can do whatever you want with the Xs, which are assumed to be constants in the model
- This allows us to capture a wide variety of non-linear (x-y) relationships
- Classic transformations
 - Logarithms
 - Polynomials
 - Splines

Polynomials (Degree 1)



Splines



Logistic Regression (8)

- Moreover, we can multiply Xs with one another, resulting in interactions
- These are powerful tools, allowing us to answer questions such as “how does the effect of X_1 change when the value of X_2 changes”?
- In parallel, estimating interactions is asking for quite a lot from your data, and overfitting becomes a much larger concern
 - A rule of thumb says you need 16 times more data to estimate interactions than to estimate main effects

Logistic Regression (9)

```
# model
fit <- glm(
  CVD ~
    SEX
  + TOTCHOL
  + AGE
  + SYSBP
  + DIABP
  + CURSMOKE
  + CIGPDAY
  + BMI
  + DIABETES
  + BPMEDS
  + HEARTRTE
  + GLUCOSE,
  data = data_lr, family = binomial(link = "logit")
)

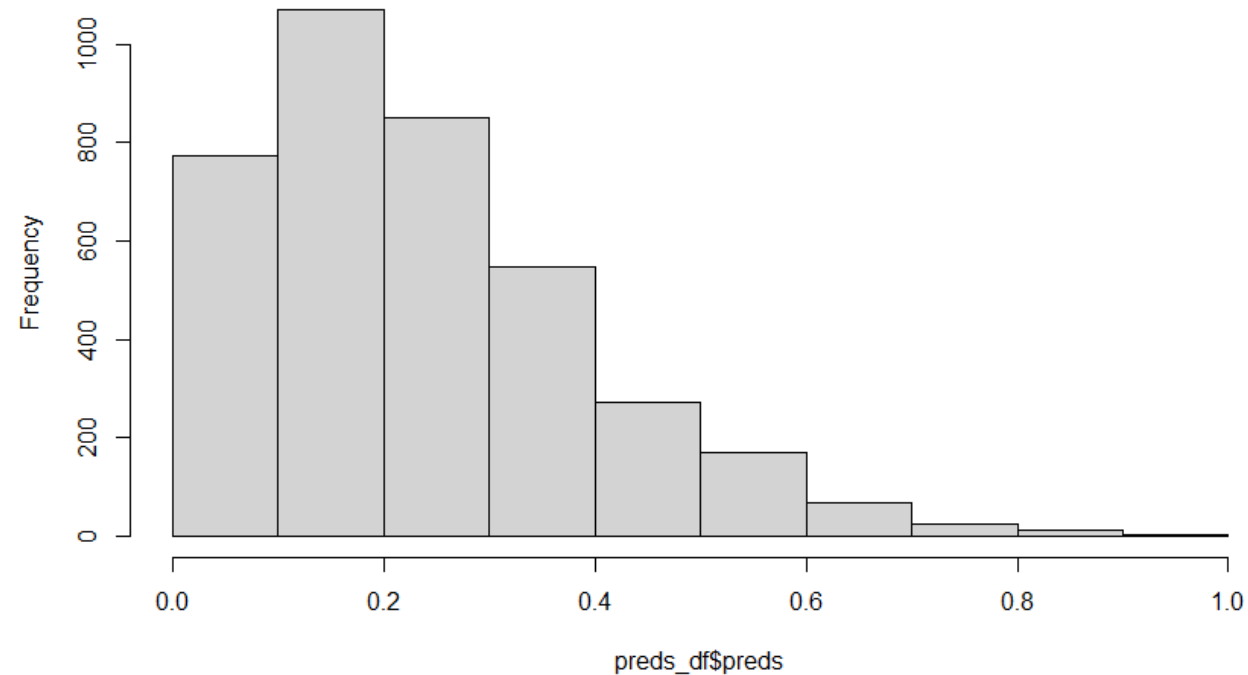
# summary
tidy(fit, exponentiate = T, conf.int = T) %>% mutate_if(is.numeric, round, 3)
```

	term <chr>	estimate <dbl>	std.error <dbl>	statistic <dbl>	p.value <dbl>	conf.low <dbl>	conf.high <dbl>
1	(Intercept)	0.001	0.531	-14.3	0	0	0.001
2	SEXfemale	0.432	0.091	-9.21	0	0.361	0.516
3	TOTCHOL	1.00	0.001	5.26	0	1.00	1.01
4	AGE	1.05	0.005	8.40	0	1.04	1.06
5	SYSBP	1.01	0.003	3.52	0	1.00	1.02
6	DIABP	1.01	0.006	2.34	0.019	1.00	1.02
7	CURSMOKEYes	1.46	0.131	2.86	0.004	1.12	1.88
8	CIGPDAY	1.00	0.005	0.354	0.723	0.991	1.01
9	BMI	1.04	0.011	3.50	0	1.02	1.06
10	DIABETESyes	2.66	0.278	3.53	0	1.55	4.62
11	BPMEDSyes	1.03	0.216	0.121	0.904	0.668	1.56
12	HEARTRTE	0.989	0.004	-3.05	0.002	0.982	0.996
13	GLUCOSE	1.00	0.002	1.64	0.102	0.999	1.01

Logistic Regression (10)

```
#####  
# preds #  
#####  
preds_df <- data.frame(labels = data_lr$CVD, preds = predict(fit, type = "response"))  
hist(preds_df$preds)
```

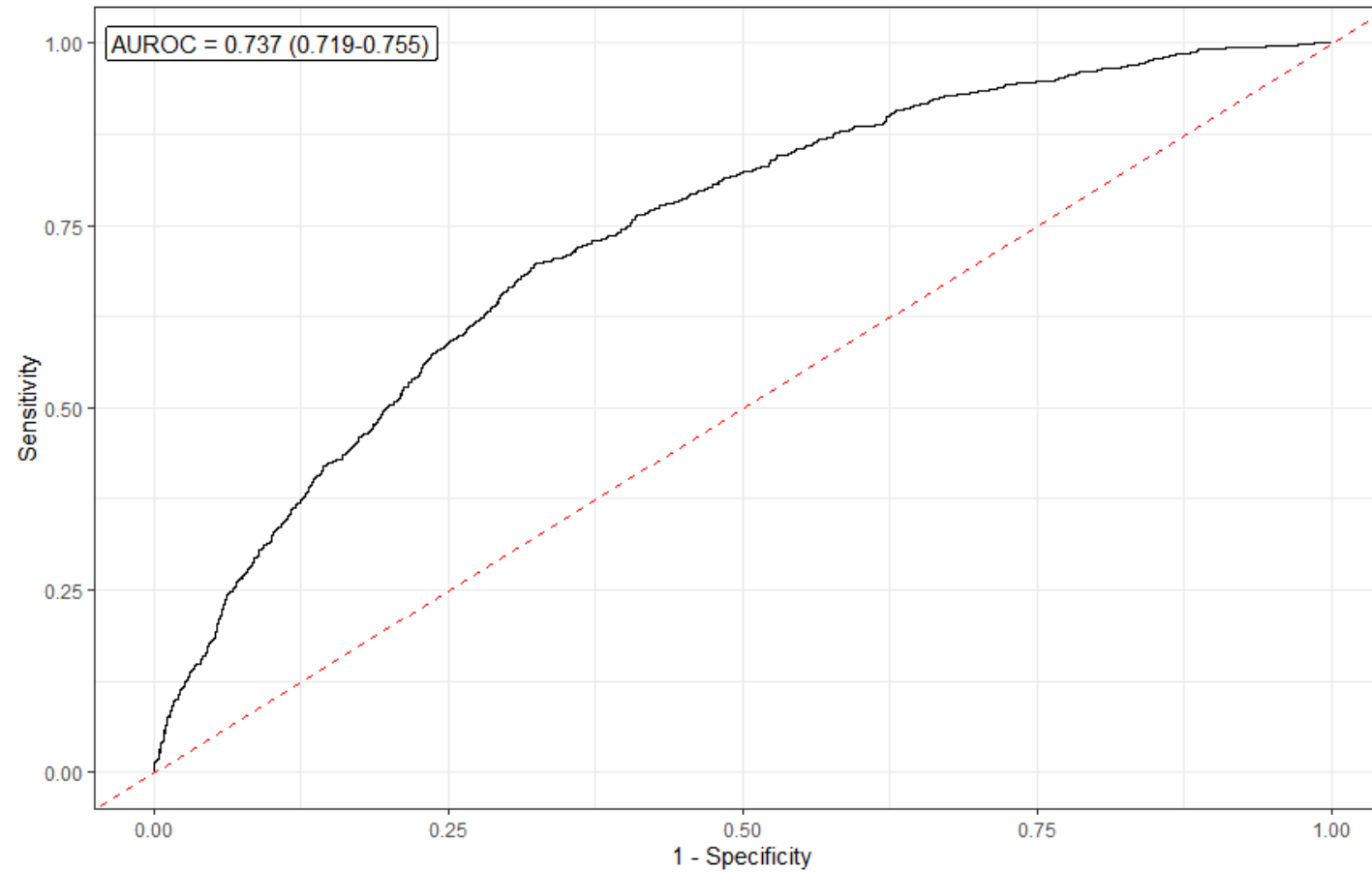
Histogram of preds_df\$preds



Logistic Regression (11)

```
#####  
# ROC #  
#####  
# run  
pROC_obj <- roc(preds_df$labels, preds_df$preds, ci=TRUE, ci.alpha=0.95)  
  
# extract  
ROC_df <- data.frame(FNR = 1 - pROC_obj$specificities, TPR = pROC_obj$sensitivities)  
ci <- glue("AUROC = {round(pROC_obj$auc[[1]],3)} ({round(pROC_obj$ci[[1]],3)}-{round(pROC_obj$ci[[3]],3)})")  
  
# plot  
ggplot(ROC_df, aes(FNR, TPR)) +  
  geom_line() +  
  theme_bw() +  
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed") +  
  xlab("1 - Specificity") +  
  ylab("Sensitivity") +  
  geom_label(aes(0.1,1), label = ci)
```

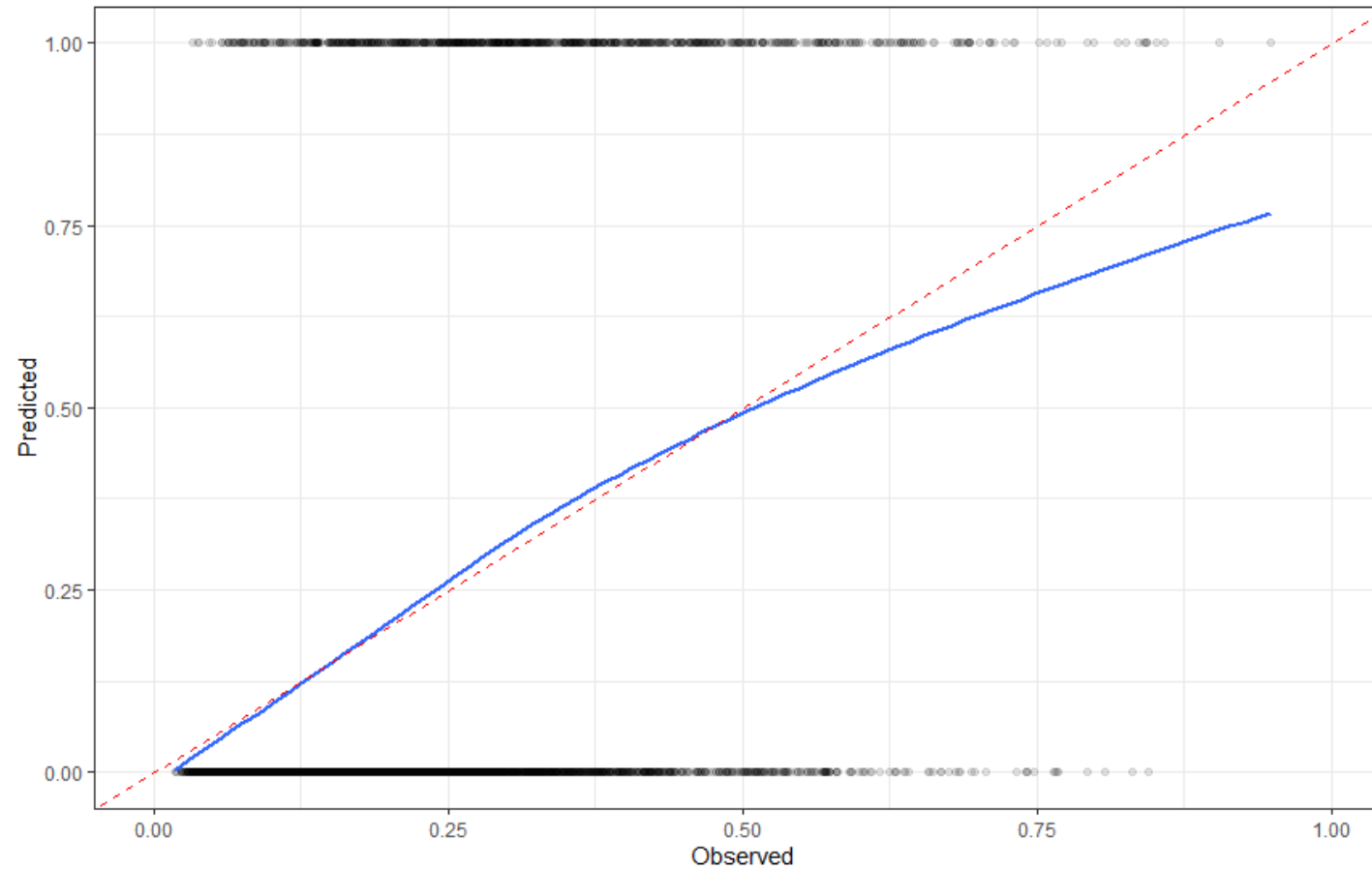
Logistic Regression (12)



Logistic Regression (13)

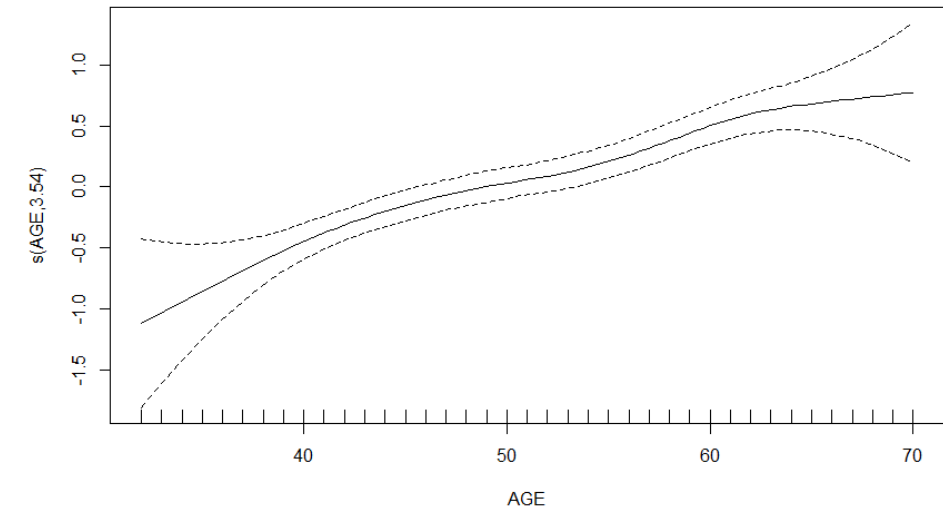
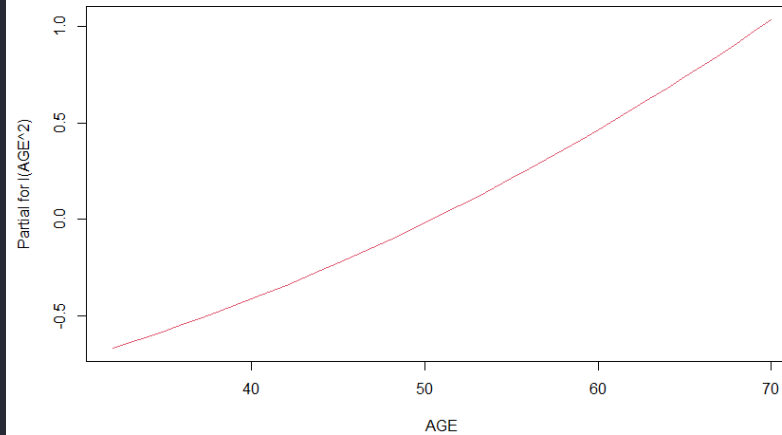
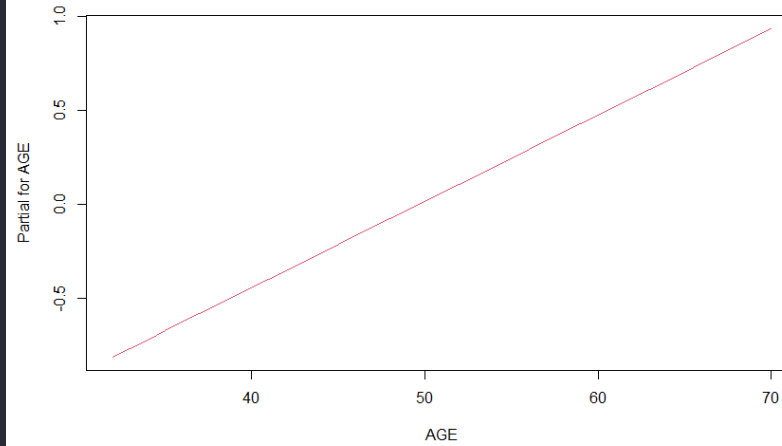
```
#####  
# Calibration #  
#####  
# plot  
ggplot(preds_df, aes(preds, labels)) +  
  geom_point(alpha = 0.1) +  
  geom_smooth(se = F) +  
  theme_bw() +  
  coord_cartesian(xlim = c(0,1), ylim = c(0,1)) +  
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed") +  
  xlab("Observed") +  
  ylab("Predicted")
```

Logistic Regression (14)



Logistic Regression (15)

```
#####  
# Terplots #  
#####  
# linear  
termplot(fit, terms = c("AGE", "SEX"))  
  
# polynomial  
fit <- glm(  
  CVD ~  
    SEX  
    + TOTCHOL  
    + I(AGE^2)  
    + SYSBP  
    + DIABP  
    + CURSMOKE  
    + CIGPDAY  
    + BMI  
    + DIABETES  
    + BPMEDS  
    + HEARTRTE  
    + GLUCOSE,  
  data = data_lr, family = binomial(link = "logit")  
)  
termplot(fit, terms = c("I(AGE^2)"))  
  
# spline  
fit <- gam(  
  CVD ~  
    SEX  
    + TOTCHOL  
    + s(AGE, bs = "cs")  
    + SYSBP  
    + DIABP  
    + CURSMOKE  
    + CIGPDAY  
    + BMI  
    + DIABETES  
    + BPMEDS  
    + HEARTRTE  
    + GLUCOSE,  
  data = data_lr, family = binomial(link = "logit")  
)  
plot(fit)
```



Logistic Regression (16)

```
# margins
fit <- glm(
  CVD ~
  SEX
+ TOTCHOL
+ AGE
+ SYSBP
+ DIABP
+ CURSMOKE
+ CIGPDAY
+ BMI
+ DIABETES
+ BPMEDS
+ HEARTRTE
+ GLUCOSE,
  data = data_lr, family = binomial(link = "logit")
)

tidy(fit, exponentiate = T, conf.int = T) %>% mutate_if(is.numeric, round, 3) %>% slice(10)

avg_comparisons(fit, variables = list(DIABETES = "reference"))
```

```
# A tibble: 1 x 7
  term      estimate std.error statistic p.value conf.low conf.high
  <chr>      <dbl>    <dbl>    <dbl>   <dbl>   <dbl>    <dbl>
1 DIABETESyes  2.66      0.278     3.53     0      1.55     4.62
```

	Term Contrast	Estimate	Std. Error	z	Pr(> z)	2.5 %	97.5 %
	DIABETES yes - no	0.181	0.0574	3.16	0.00157	0.0689	0.294

Multinomial Regression

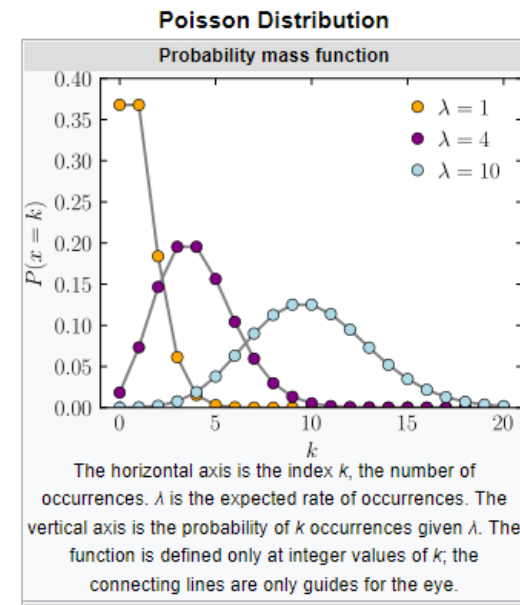
- It is easy to extend logistic regression to a situation where the outcome has more than 2 unordered levels
- This is called multinomial (softmax) regression
- One of the outcomes is chosen as the baseline
- different coefficients are estimated for each outcome except the baseline
- The exponents of the coefficients are the odds ratio compared to the baseline level
- The prediction for a given outcome is given as $\frac{e^{\beta_{k0} + \beta_{k1}x_1 + \dots + \beta_{kp}x_p}}{1 + \sum_{l=1}^{K-1} e^{\beta_{l0} + \beta_{l1}x_1 + \dots + \beta_{lp}x_p}}$

Poisson Regression

- Logistic regression is a great tool
 - Interpretable
 - Capable of modeling non-linear relationships with transformations of the predictors
 - Good predictive performance, particularly with regularization
- But it has two big limitation
 - It only works with categorical outcomes
 - It does not take time into account, requiring “flat” data
 - Time is external to the model, e.g., we model “heart attack in the next 10 years”
 - Specifically, this means that there is no way to model individuals with different follow-up times

Poisson Regression (2)

$$Y \sim \text{Poisson}(\lambda)$$
$$\log(\lambda) = \sum_{i=1}^n \beta_i x_i$$



- The distribution is Poisson, and the link function is the natural logarithm
- The support of the Poisson distribution, as you well recall, is the non-negative integers
- has a single parameter, λ , which describes the expected rate of occurrences
- It is the limit of the binomial distribution when $n \rightarrow \infty$, $p \rightarrow 0$

Poisson Regression (3)

- Accordingly, the basic use of Poisson regression is when the outcome variable is a count of something
- For example, how many hospitalizations someone will experience over the next 5 years
- This is in fact not very common, and if that were all Poisson regression could be used for, it would probably not be in this presentation

Poisson Regression (4)

- The usefulness of Poisson regression greatly increases owing to a little algebraic trick

$$\log(\lambda) = \sum_{i=1}^n \beta_i x_i + \log(\text{time}) \Rightarrow$$

$$\log\left(\frac{\lambda}{\text{time}}\right) = \sum_{i=1}^n \beta_i x_i$$

- By including the log of the denominator (usually time) as one of the predictors (without a coefficient), we can move from modeling a count of the events to directly modeling a rate
- In statistical software, this is called the “offset”

Poisson Regression (5)

- This trick is what makes Poisson regression so popular
- Now we can model a situation in which every person has a different follow-up time
- If you recall the study design lecture, this is the regression analog of incidence rates
- Under this formulation, it's easy to see that the exponent of the coefficient is the incidence rate ratio (IRR), a good and interpretable measure of relative risk

Poisson Regression (6)

- This trick also allows a given person to “contribute” time to different exposures
- For example, if someone is unvaccinated for 10 days
- Then is vaccinated and followed-up for 20 days more and then infected
- We would analyze this dataset, and it would work well

Age	Sex	Exposure	Offset	Outcome
37	M	Unvaccinated	Log(10)	0 (no infection)
37	M	Vaccinated	Log(20)	1 (infection)

Poisson Regression (7)

- But wait, infection is 0/1, and the support for Poisson is all positive integers
 - Our model is wrong!
- It is, but Poisson regression has one more trick up its sleeve
- Modified Poisson regression is when we use Poisson regression with a binary outcome
- It requires an adjustment for the standard error, so use the proper software, otherwise you'll get wrong standard errors
- It was **massively** used for Covid-19 vaccine effectiveness studies
- (This is another example of the “any good estimator will do” approach)

Poisson Regression (8)

```
# model
fit_pois <- glm(
  CVD ~
    SEX
  + TOTCHOL
  + AGE
  + SYSBP
  + DIABP
  + CURSMOKE
  + CIGPDAY
  + BMI
  + DIABETES
  + BPMEDS
  + HEARTRTE
  + GLUCOSE,
  data = data_pois, family = poisson(link = "log"), offset = log(TIMECVD)
)

# summary
?tidy(fit_pois, exponentiate = T, conf.int = T) %>% mutate_if(is.numeric, round, 3)
```

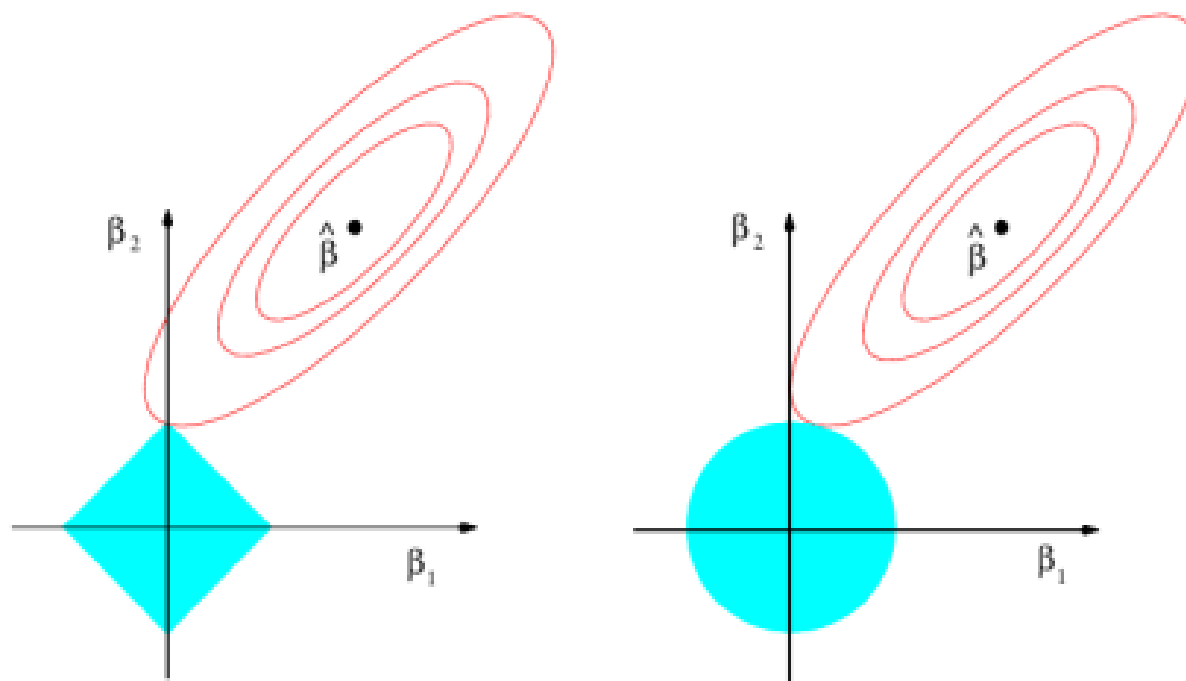
term	estimate	std.error	statistic	p.value	conf.low	conf.high
<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1 (Intercept)	0	0.284	-55.7	0	0	0
2 SEXfemale	0.454	0.047	-16.7	0	0.413	0.498
3 TOTCHOL	1.00	0	7.06	0	1.00	1.00
4 AGE	1.04	0.003	14.2	0	1.03	1.05
5 SYSBP	1.01	0.001	6.87	0	1.01	1.01
6 DIABP	1.01	0.003	3.15	0.002	1.00	1.01
7 CURSMOKEyes	1.43	0.07	5.12	0	1.25	1.64
8 CIGPDAY	0.999	0.003	-0.276	0.783	0.994	1.00
9 BMI	1.02	0.006	4.53	0	1.01	1.04
10 DIABETESyes	1.82	0.091	6.56	0	1.51	2.16
11 BPMEDSyes	1.25	0.069	3.20	0.001	1.09	1.42
12 HEARTRTE	0.996	0.002	-1.95	0.052	0.993	1
13 GLUCOSE	1.00	0.001	2.15	0.031	1	1.00

term	estimate	std.error	statistic	p.value	conf.low	conf.high
<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1 (Intercept)	0.001	0.531	-14.3	0	0	0.001
2 SEXfemale	0.432	0.091	-9.21	0	0.361	0.516
3 TOTCHOL	1.00	0.001	5.26	0	1.00	1.01
4 AGE	1.05	0.005	8.40	0	1.04	1.06
5 SYSBP	1.01	0.003	3.52	0	1.00	1.02
6 DIABP	1.01	0.006	2.34	0.019	1.00	1.02
7 CURSMOKEyes	1.46	0.131	2.86	0.004	1.12	1.88
8 CIGPDAY	1.00	0.005	0.354	0.723	0.991	1.01
9 BMI	1.04	0.011	3.50	0	1.02	1.06
10 DIABETESyes	2.66	0.278	3.53	0	1.55	4.62
11 BPMEDSyes	1.03	0.216	0.121	0.904	0.668	1.56
12 HEARTRTE	0.989	0.004	-3.05	0.002	0.982	0.996
13 GLUCOSE	1.00	0.002	1.64	0.102	0.999	1.01

Regularized Regression

- By adding certain factors to the objective function being optimized (the likelihood), we can make the coefficient estimates converge to more favorable values
- Technically, we get biased estimates with less variance
- Empirically, we get better predictions and sometimes variable selection
 - Ridge regression: $\dots + \lambda \sum_{i=1}^n \beta_i^2$
 - LASSO: $\dots + \lambda \sum_{i=1}^n |\beta_i|$
 - Elastic Net: $\dots + \lambda \left[\alpha \sum_{i=1}^n |\beta_i| + (1 - \alpha) \sum_{i=1}^n \beta_i^2 \right]$
- Where lambda and alpha are hyper-parameters to be tuned

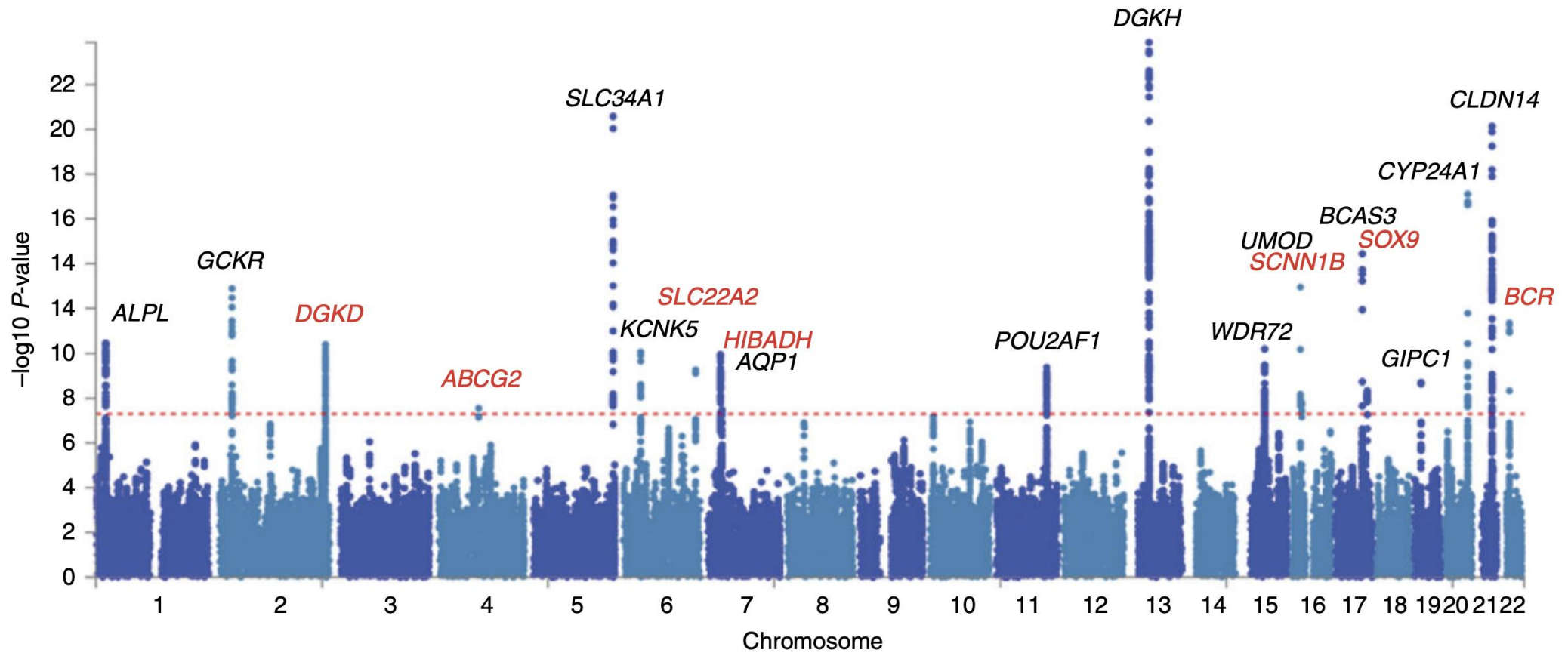
Regularized Regression (2)



Regularized Regression (3)

- LASSO, in particular, is very useful and important to be acquainted with
- It has become the standard methodology to handle sparse data, and performs well even in $p > n$ situations
 - e.g., GWAS, where you try to find an association between a clinical phenomenon and genetic variants all across the genome
- However, it is not a panacea, and does not perform so well when $p \gg n$
 - there are other feature selection methods that work in so-called extreme high-dimensionality, but we won't go into those

Regularized Regression (4)



Regularized Regression (5)

```
x <-  
  data_lr %>%  
  select(SEX:GLUCOSE) %>%  
  model.matrix(~., data = .)  
  
y <- data_lr$CVD  
  
fit_cv <- cv.glmnet(x = x, y = y, family = "binomial", type.measure = "auc")  
print(fit_cv)  
coef(fit_cv)
```

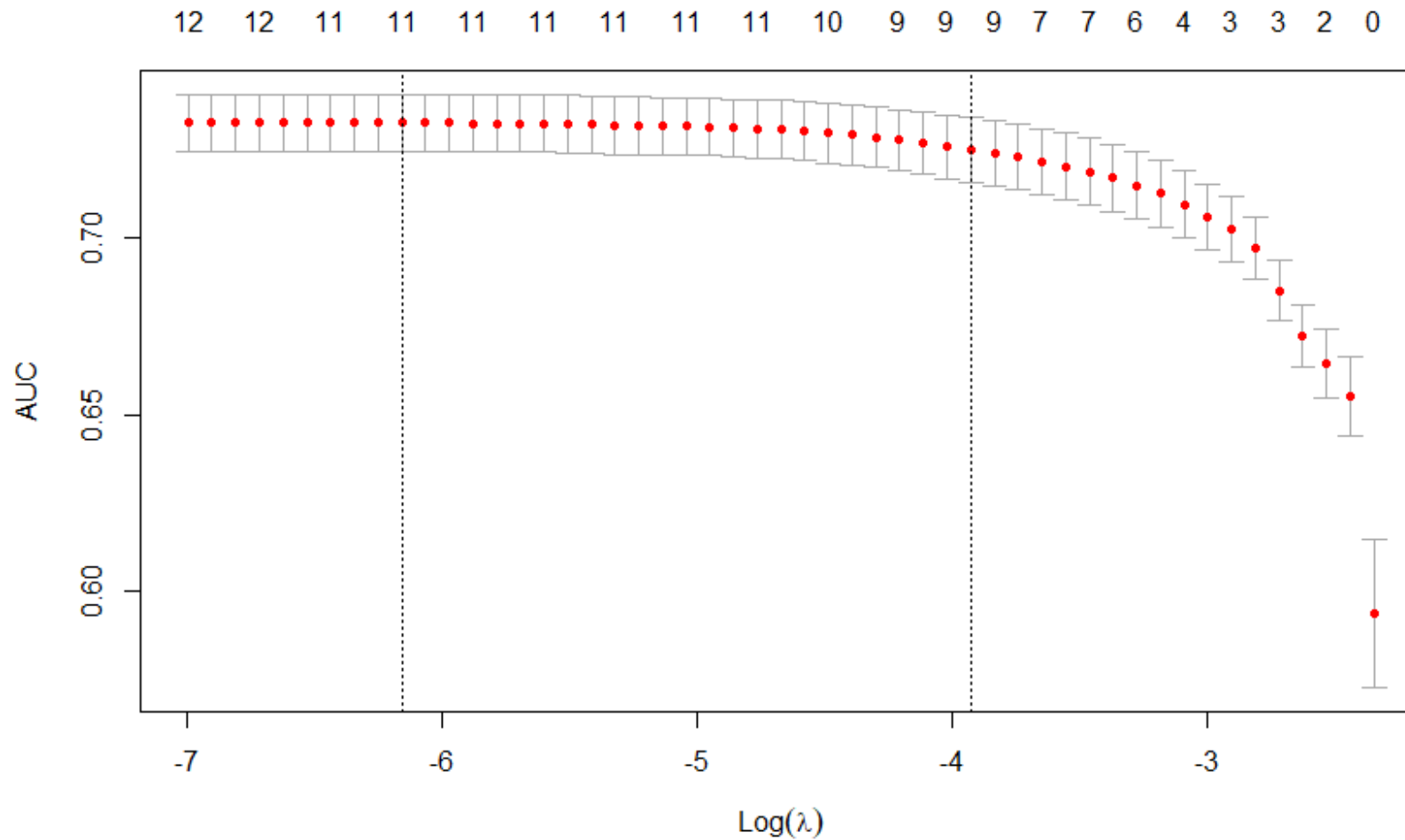
Measure: AUC

	Lambda	Index	Measure	SE	Nonzero
min	0.003064	38	0.7324	0.008370	11
1se	0.021619	17	0.7250	0.007307	9

	s1
(Intercept)	-5.096775e+00
(Intercept)	.
SEXfemale	-6.148815e-01
TOTCHOL	2.195494e-03
AGE	3.019961e-02
SYSBP	1.036543e-02
DIABP	5.423500e-03
CURSMOKEyes	2.667770e-02
CIGPDAY	.
BMI	1.156007e-02
DIABETESyes	6.940354e-01
BPMEDSyes	.
HEARTRTE	.
GLUCOSE	5.814365e-05

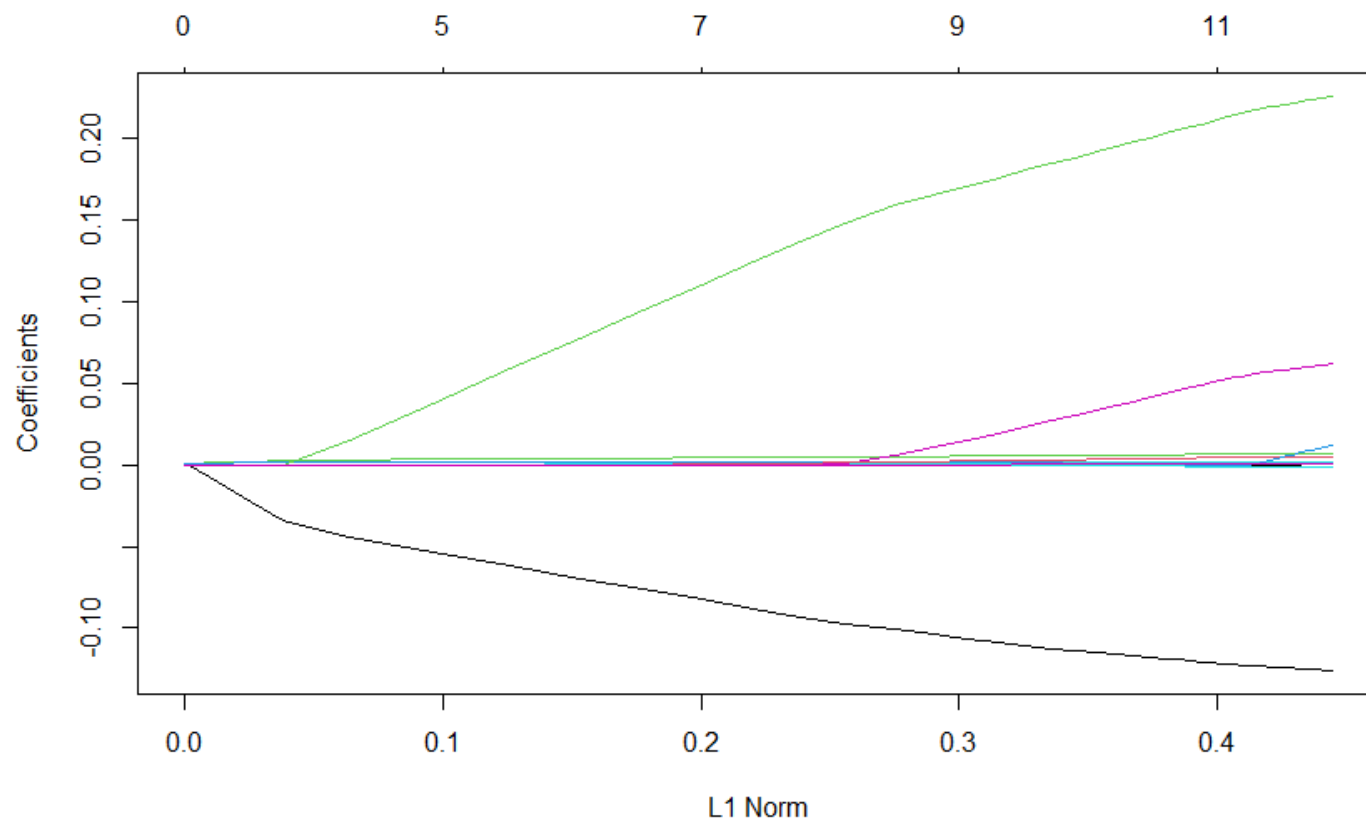
Regularized Regression (6)

```
plot(fit_cv)
```



Regularized Regression (7)

```
plot(fit_cv$glmnet.fit)
```



Ensembles of Decision Trees (ever so briefly)

- Decision trees have many advantages (interpretable, invariant to scaling), but have high variance
- Ensembles of trees tend to outperform single trees, at the cost of computation and less interpretability
- Random forests (Breiman 2001) create each tree using a bootstrap sample from the original training set, at each node considering only a subset of the features for splitting
- Boosting specializes each consecutive tree to the samples on which previous trees erred
- Gradient boosted decision trees tend to give the best “out-of-the-box” performance on structured data, including (surprisingly) time series data

Survival Analysis – Introduction

(Based on Prof. Malka Gorfine's TAU Class)

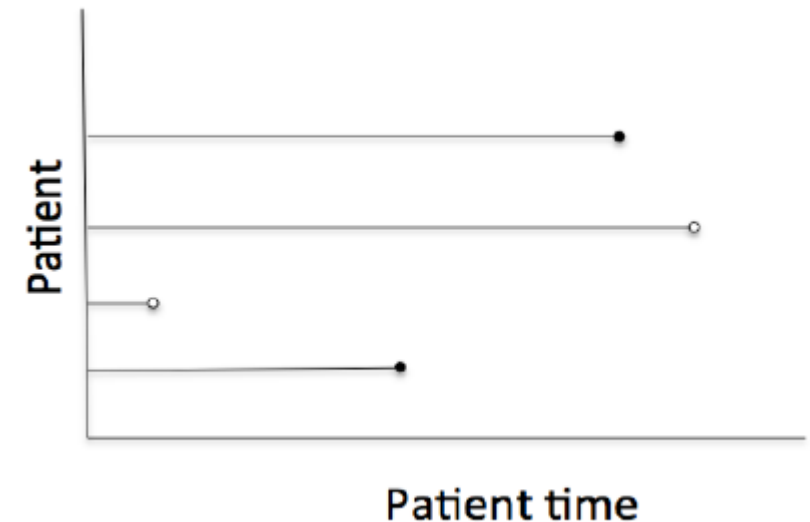
- Medical data very often uses time-to-event outcomes
 - Time until disease progression
 - Time to death
 - Etc.
- These are outcomes composed of both an indicator for the event (yes/no) and the time in which the patient was under surveillance
- These very often come with censoring and/or truncation

Censoring & Truncation

- Censoring – We only know that the event happened at a certain range (missing data)
 - Right Censoring – we know that the event happened later than some time
 - Left Censoring – we know that the event happened before some time
 - Interval Censoring – we know that the event happened during some interval
- Truncation – Certain event times lead to the patient not being recorded (selection bias)
 - Left Truncation – patients who have experienced the event before some time are not recorded
 - Right Truncation – patients who have experienced the event after some time are not recorded

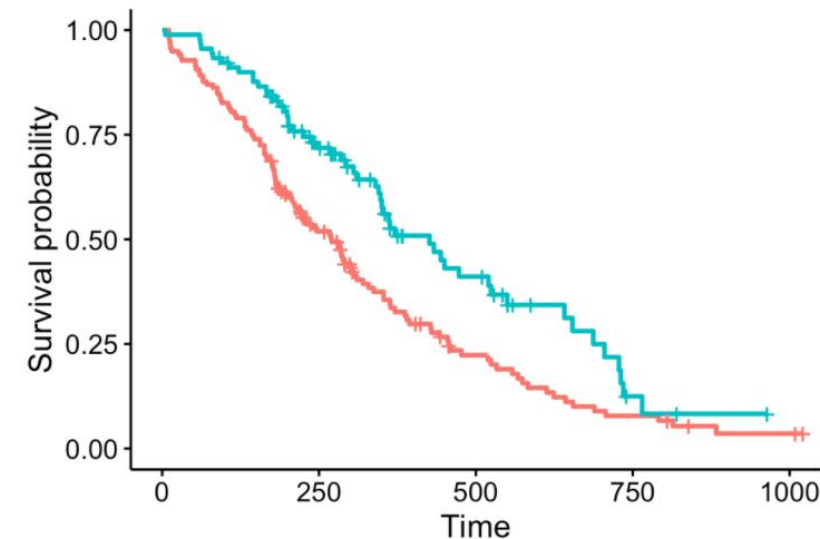
Time-to-Event Outcomes

- The most standard form is that of right censoring alone
 - Less common is left truncation + right censoring
 - Anything else is “fancy”
- So, we have pairs of (T, δ) , where T is the last time of follow-up and δ is an indicator of whether an event was observed up until that time
- We could have
 - $(20, 0)$ – observed until time 20 and then censored
 - $(14, 1)$ – observed until time 14 and then experienced an event



Why Survival Analysis

- Several big reasons
 - Time matters. Consider these two curves. If we just looked at survival until time $T=750$ they would look identical, but they are really very different
 - We want to make use of all the information we have, including patients only observed some of the time
 - If censoring is not random, then ignoring the censored will bias the results



Basic Quantities (the discrete case)

- The basic random variable of interest is X , a non-negative RV representing the time until an event occurs
- It is commonly described using one of three functions, where each uniquely determines the others
 - Mass: $f(x) = P(X = x)$
 - Survival: $S(x) = P(X > x) = 1 - F(x) = 1 - P(X \leq x) = \sum_{x_j > x} P(X = x_j) = \sum_{x_j > x} f(x_j)$
 - a non-increasing step function
 - Hazard: $h(x) = P(X = x | X \geq x) = \frac{P(X=x)}{S(x-1)}$

The Kaplan-Meier Estimator

- We want the survival probability at time x , but can't just calculate $P(X > x)$ because of censoring
- But we have

$$h(x) = \frac{P(X = x)}{S(x-1)} = \frac{S(x-1) - S(x)}{S(x-1)} = 1 - \frac{S(x)}{S(x-1)}$$

$$S(x) = \prod_{x_j \leq x} \frac{S(x_j)}{S(x_{j-1})} = \prod_{x_j \leq x} [1 - h(x_j)]$$

- So, defining the number at risk at each time point as y_{x_j} and the number who experienced an event as d_{x_j} , we have that

$$\hat{S}(x) = \prod_{x_j \leq x} \left(1 - \frac{d_{x_j}}{y_{x_j}} \right)$$

The Kaplan-Meier Estimator (2)

- This sounds more complicated than it is
- We can't estimate the survival directly, but we can estimate the hazard at each time point an event occurs
- So, we multiply one minus the hazard at each time point
- The resulting function is a non-increasing step function, with a jump at each observed failure time
- In essence, those that remain stand for those that left, so we are assuming non-informative censoring
- A standard error can be had using Greenwood's formula

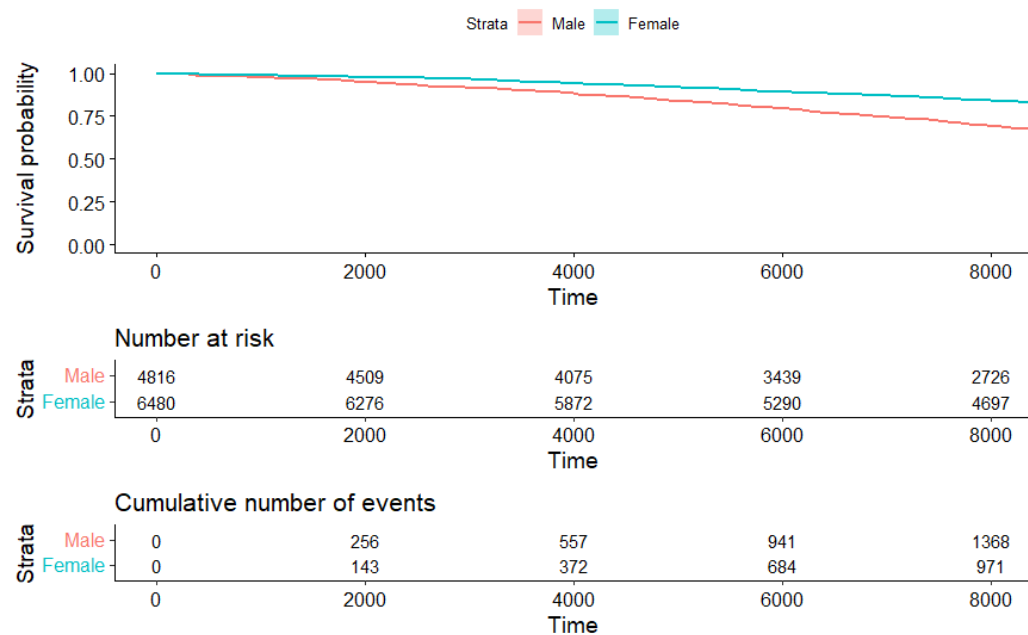
The Kaplan-Meier Estimator (3)

```
#####  
# KM #  
#####  
fit_KM <- survfit(Surv(TIMECVD, CVD) ~ SEX, data = data)  
|
```

```
> tidy(fit_KM) %>% filter(time < 150) %>% print(n=20)  
# A tibble: 12 x 9  
   time n.risk n.event n.censor estimate std.error conf.high conf.low strata  
   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <chr>  
1    26  4816     1     0    1.00  0.000208     1    0.999 SEX=ma  
2    27  4815     3     0    0.999  0.000415     1    0.998 SEX=ma  
3    87  4812     0     1    0.999  0.000415     1    0.998 SEX=ma  
4   108  4811     3     0    0.999  0.000550     1    0.997 SEX=ma  
5    47  6480     1     0    1.00  0.000154     1    1.00  SEX=fema  
6    58  6479     0     1    1.00  0.000154     1    1.00  SEX=fema  
7    73  6478     0     1    1.00  0.000154     1    1.00  SEX=fema  
8   101  6477     3     0    0.999  0.000309     1    0.999 SEX=fema  
9   110  6474     3     0    0.999  0.000409     1    0.998 SEX=fema  
10  126  6471     0     1    0.999  0.000409     1    0.998 SEX=fema  
11  145  6470     0     1    0.999  0.000409     1    0.998 SEX=fema  
12  146  6469     0     1    0.999  0.000409     1    0.998 SEX=fema
```

The Kaplan-Meier Estimator (4)

```
ggsurvplot(  
  fit = fit_KM  
  ,data = data  
  ,cumevents = T  
  ,risk.table = T  
  ,censor = F  
  ,conf.int = T  
  | ,fontsize = 4  
  ,legend.labs = c("Male", "Female")  
)
```



The Log-rank Test

- We want to know if two KM plots differ
- $H_0: h_0(t)=h_1(t)$, for all t
- Under the null, at each time i , given the number at risk and number of events in each group, the number of events in a specific group has a hypergeometric distribution, for which we know the expectation (O_i) and variance (V_i)
- The statistic is then:
$$Z = \frac{\sum_{i=1}^T (O_i - E_i)}{\sqrt{(\sum_{i=1}^T V_i)}}$$
- And by the CLT it converges to a normal standard distribution
- Later we will view $|Z|$ as a measure of separation of the survival curves

The Log-rank Test (2)

```
#####  
# LogRank #  
#####  
survdif(Surv(TIMECVD, CVD) ~ SEX, data = data)
```

```
Call:  
survdif(formula = Surv(TIMECVD, CVD) ~ SEX, data = data)
```

	N	Observed	Expected	(O-E)^2/E	(O-E)^2/v
SEX=male	4816	1483	1021	209	347
SEX=female	6480	1085	1547	138	347

Chisq= 347 on 1 degrees of freedom, p= <2e-16

Parametric Models

- There are many popular parametric models, which we won't get into
- For example, the exponential model

$$S(x) = \exp(-\lambda x)$$

$$f(x) = \lambda \exp(-\lambda x)$$

$$h(x) = \lambda$$

- And we get a (rather) weird constant hazard and no memory

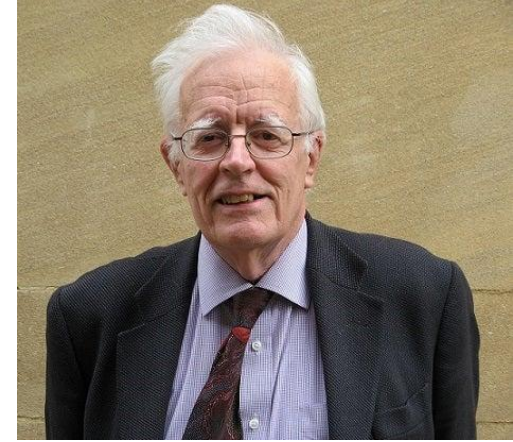
$$P(X \geq x + z | X \geq x) = P(X \geq z)$$

- Biologically this usually doesn't make much sense, but lightbulbs, I think, obey this...

Regression Models

- We want to incorporate a vector of features/predictors, Z , into the model
- The variables in Z can be categorical or continuous
- They can either be time-constant (e.g., sex) or time-varying (e.g., laboratory values)
- Two popular approaches
 - Accelerated Failure Time (AFT) models
 - Cox Proportion Hazards Regression (or just Cox regression)

Cox Regression



- Instead of modeling the survival, we model the hazard

$$h(x|Z) = h_0(x) \exp(\beta^T Z)$$

- We get proportional hazards that are independent of time. Assume Z_1 and Z_2 differ only in one unit of the variable z_1

$$\frac{h(x|Z_1)}{h(x|Z_2)} = \frac{\exp(\beta^T Z_1)}{\exp(\beta^T Z_2)} = \frac{\exp(\beta_1(z_1 + 1))}{\exp(\beta_1(z_1))} = \exp(\beta_1)$$

- So, the exponent of each coefficient is the hazard ratio (HR)
- The baseline hazard remains unspecified, making this a semi-parametric model

Cox Regression (2)

- How does the data look?

ID	Age	Sex	Exposure	Time	Event
1	37	M	Unvaccinated	10	0 (Censored)
2	37	M	Vaccinated	12	1 (Infection)

Partial Likelihood

- Censoring prevents us from writing out a standard likelihood function (i.e., we don't know the event time for everyone), so we use the partial likelihood
- The partial likelihood looks at the individuals who experienced an event at each time out of those still at risk at that time

$$PL(\beta) = \prod_{i \in D} \frac{h_0(X_i) \exp(\beta^T Z_i)}{\sum_{j \in R(X_i)} h_0(X_j) \exp(\beta^T Z_j)}$$

- Where D is the set of patients for which I witness an event, and $R(x)$ is the risk set at time $X=x$
- This (admittedly weird) estimator can be shown to be unbiased and consistently asymptotically normal

Assumptions

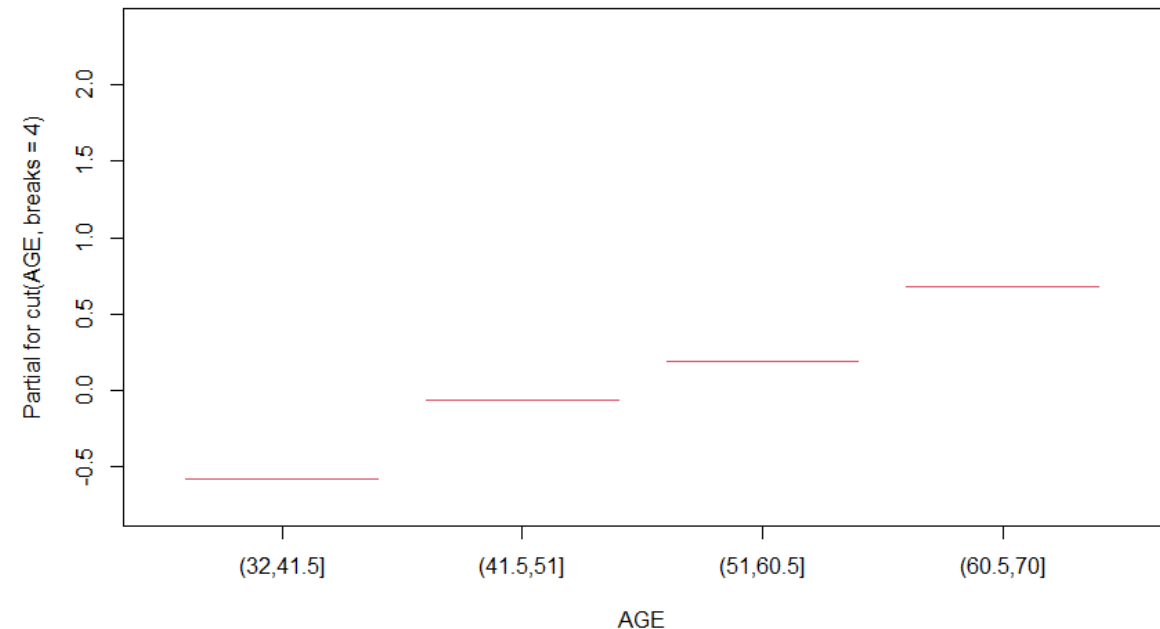
- The two main testable assumptions in a Cox model are
 - Linearity of effects
 - Proportional hazards

Assumptions (2)

- When we enter a continuous predictor into a regression, we generally have 3 options
 - Categorize it and estimate a different coefficient for each level (ages 20-29, 30-39, etc.)
 - Add the variable in its natural form, and assume linearity in the linear predictor scale (e.g., in a Cox regression, linearity in the hazard)
 - Add the variable in a flexible form (fractional polynomials, splines, etc.)

Assumptions (3)

- Categorization is controversial
- On the one hand, we get interpretable coefficients (being 20-29 increases risk by 2, being 30-39 by 2.5, etc.)
- On the other hand
 - We enforce a non-biological step-function form on the relationship
 - We add a lot of degrees of freedom
 - We lose a lot of information
- Epidemiologists tend to favor categorization. Statisticians don't



Assumptions (4)

- Using a flexible form is considered the best way, but
 - sacrifices interpretability (there's no longer a single effect)
 - and increases the expressivity of the model (so higher risk for overfitting)
- If we did use a linear effect, there are different ways to check the linearity assumption, the simplest of which is to also fit with a flexible form and see how “non-linear” it was estimated to be

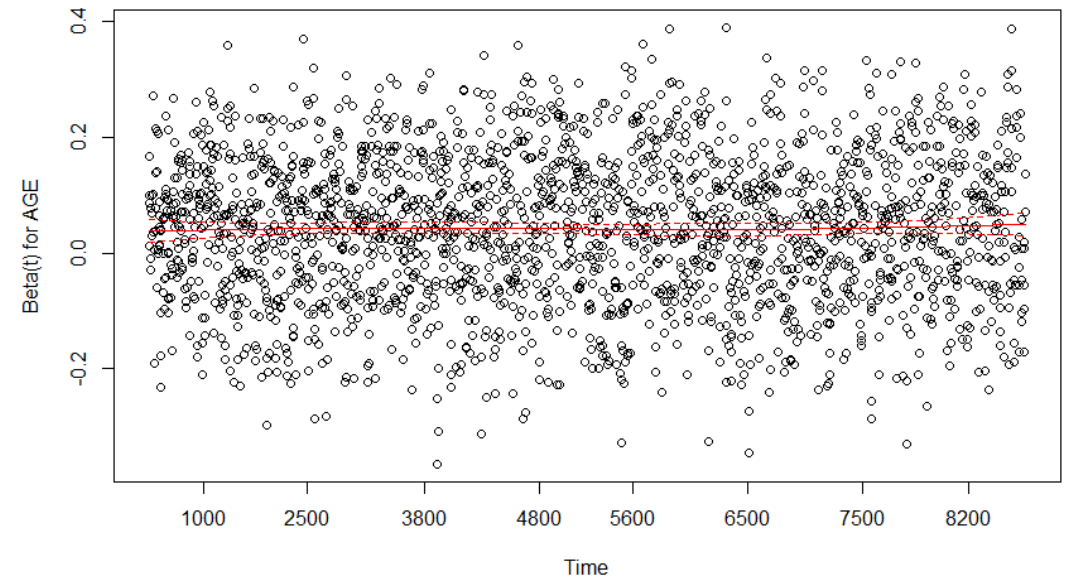
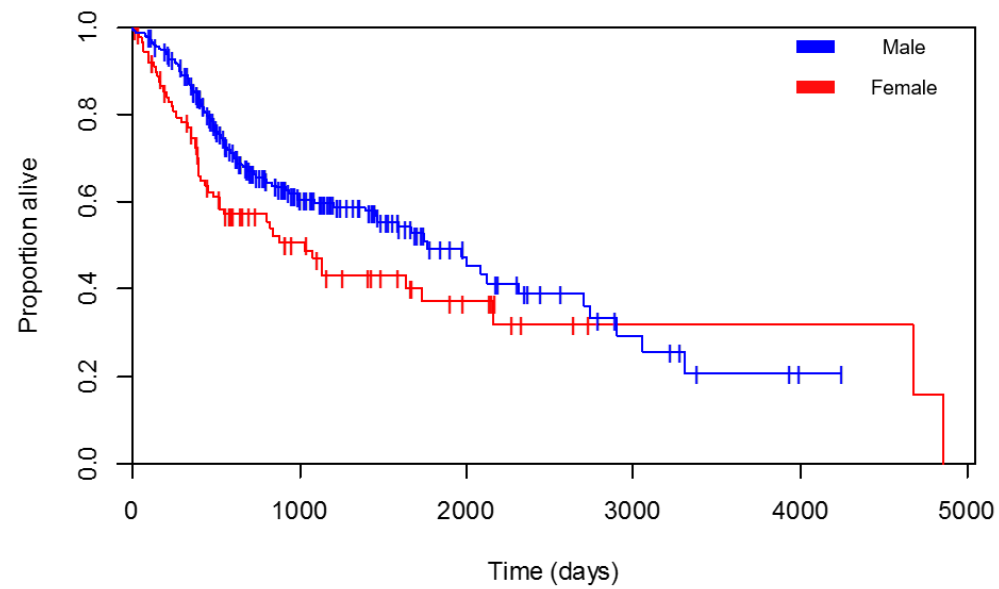
Assumptions (5)

- Proportionality is the more unique and problematic assumption, and the one people usually care about
 - The model is called Cox PROPORTIONAL hazards
- Depending on the time scale, it is also often not very plausible
 - Severity of the disease could be more important early on
 - Things have different effects depending on age
 - Calendar time changes a lot, often including effects of things

Assumptions (6)

- The assumption itself is usually tested either
 - Crudely, by looking at KM plots and seeing if they intersect
 - By looking for a pattern in the Schoenfeld residuals for each variable

Assumptions (7)



Assumptions (8)

- Remember, we are dealing with prediction, so (to a first approximation at least) we don't inherently care about violations of assumptions
 - If the performance is good, we are happy
- Regardless, violations of the assumptions can direct us to ways in which we can improve the model's fit and performance
- If we intend to make any interpretation of the coefficients, however, the assumptions are critical

Cox Regression (3)

```
#####  
# Cox #  
#####  
# complete cases  
data_cox <- data[complete.cases(data),]  
  
# model  
fit_cox <- coxph(  
  Surv(TIMECVD, CVD) ~  
    SEX  
  + TOTCHOL  
  + AGE  
  + SYSBP  
  + DIABP  
  + CURSMOKE  
  + CIGPDAY  
  + BMI  
  + DIABETES  
  + BPMEDS  
  + HEARTRTE  
  + GLUCOSE,  
  data = data_cox, model = T, x = T, y = T  
)  
  
# summary  
tidy(fit_cox, exponentiate = T, conf.int = T) %>% mutate_if(is.numeric, round, 3)
```

Cox Regression (4)

```
# A tibble: 12 x 7
```

	term <chr>	estimate <dbl>	std.error <dbl>	statistic <dbl>	p.value <dbl>	conf.low <dbl>	conf.high <dbl>
1	SEXfemale	0.435	0.048	-17.5	0	0.396	0.478
2	TOTCHOL	1.00	0	7.16	0	1.00	1.00
3	AGE	1.04	0.003	15.2	0	1.04	1.05
4	SYSBP	1.01	0.001	7.48	0	1.01	1.01
5	DIABP	1.01	0.003	3.33	0.001	1.00	1.01
6	CURSMOKEyes	1.48	0.069	5.62	0	1.29	1.69
7	CIGPDAY	0.999	0.003	-0.452	0.651	0.993	1.00
8	BMI	1.02	0.006	4.52	0	1.01	1.04
9	DIABETESyes	1.91	0.091	7.14	0	1.60	2.29
10	BPMEDSyes	1.26	0.069	3.35	0.001	1.10	1.44
11	HEARTRTE	0.996	0.002	-1.94	0.053	0.993	1
12	GLUCOSE	1.00	0.001	2.14	0.033	1	1.00

Extension 1 – Time Varying Covariates

- When Z is a function of time, things become more complicated for prediction
- Estimation is “easy”, Cox regression can handle this situation naturally, because it looks at each time point independently

Extension 1 – Time Varying Covariates (2)

- How would the data look?
- We need to move to the “counting process” formulation
- (time_end, event) => (time_start, time_end, event)

ID	Age	Sex	LDL	Time Start	Time End	Outcome
1	37	M	80	0	10	0
1	37	M	110	11	30	1

Extension 1 – Time Varying Covariates (3)

- But how then do you predict an event from time $T=t$ up to time $T=s$, if you don't know how the covariates will behave between times t and s ?
- Two options:
 - Landmarking - You can just sum up everything you know until time t and hope for the best
 - Joint modeling - You can try to model the way the covariates behave together with the hazard function

Extension 2 – Other Time Scales

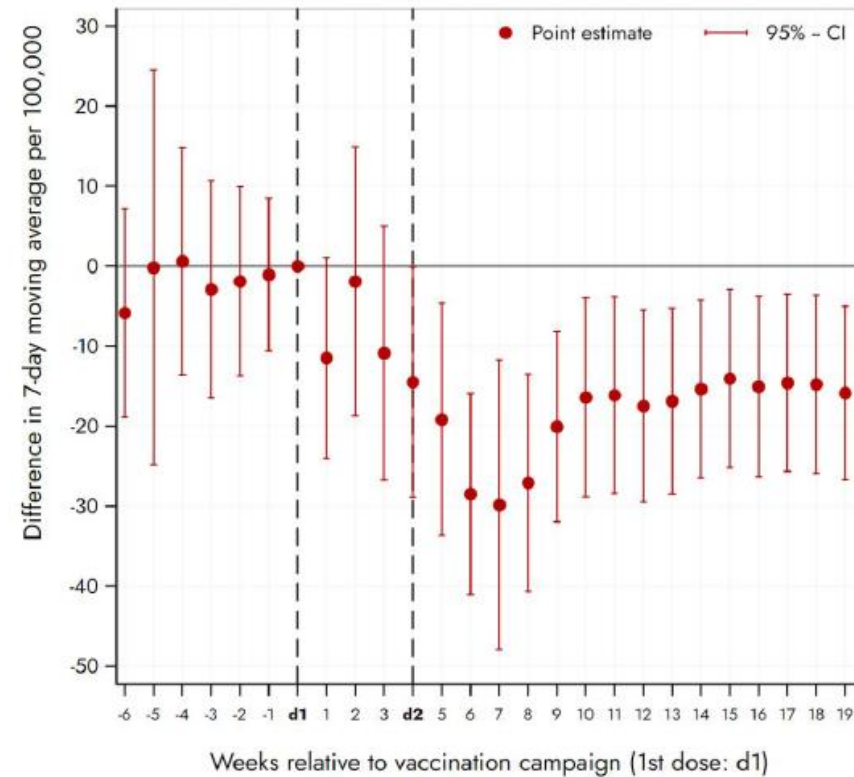
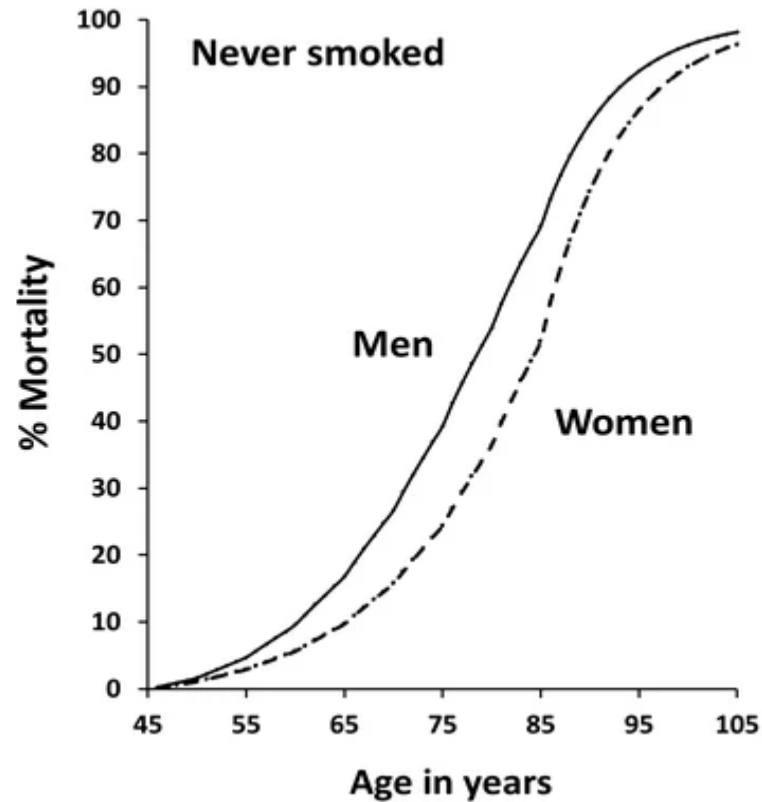
- Usually, the time part of the time-to-event outcome is time-on-trial (time since someone started a new drug, was vaccinated, etc.)
- But we can instead use other time scales such as calendar time or age
- This would capture, in the nonparametric part of the model, all the variance resulting from those variables, leaving the parametric part to handle the rest, which we care about
- Very useful when time-on-trial doesn't really make sense, two examples:
 - In cardiovascular disease prediction models, using age
 - In vaccine effectiveness studies, using calendar time

Extension 2 – Other Time Scales (2)

- How would the data look?

ID	Sex	LDL	Age Start	Age End	Outcome
1	M	80	30	39	0
1	M	110	40	49	1

Extension 2 – Other Time Scales (3)



Extension 3 – Competing Risks

- Often, we encounter a situation where more than one outcome is possible, and each outcome is mutually exclusive
- For example, we want the time until a heart attack, but some people die of alternate reasons
- If we just treat them as censored, we would overestimate the probability of the outcome (because they would still be considered at risk)
- So, this is a sort of very informative censoring, and must be handled using special means

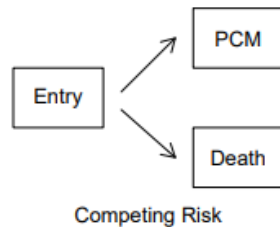
Extension 3 – Competing Risks (2)

- How would the data look?

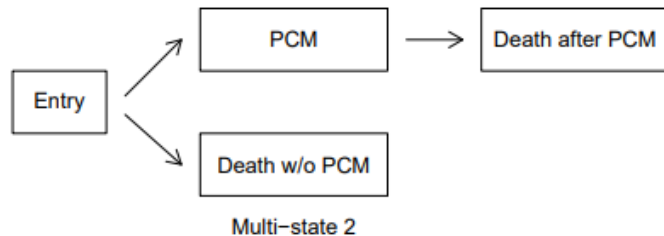
ID	Age	Sex	Time	Outcome
1	45	M	10	0 (Censored)
2	37	M	8	1 (Infection)
3	75	F	5	2 (Death from other causes)

Extension 3 – Competing Risks (3)

- Competing risks is the most basic form of a multistate model

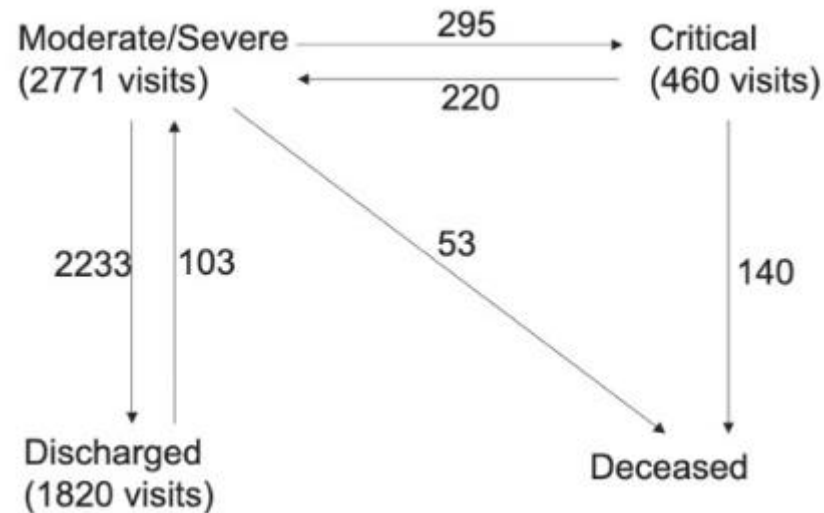


- But multistate models can become arbitrarily complicated, modeling transitions back and forth between any number of states using the machinery of survival analysis



Extension 3 – Competing Risks (4)

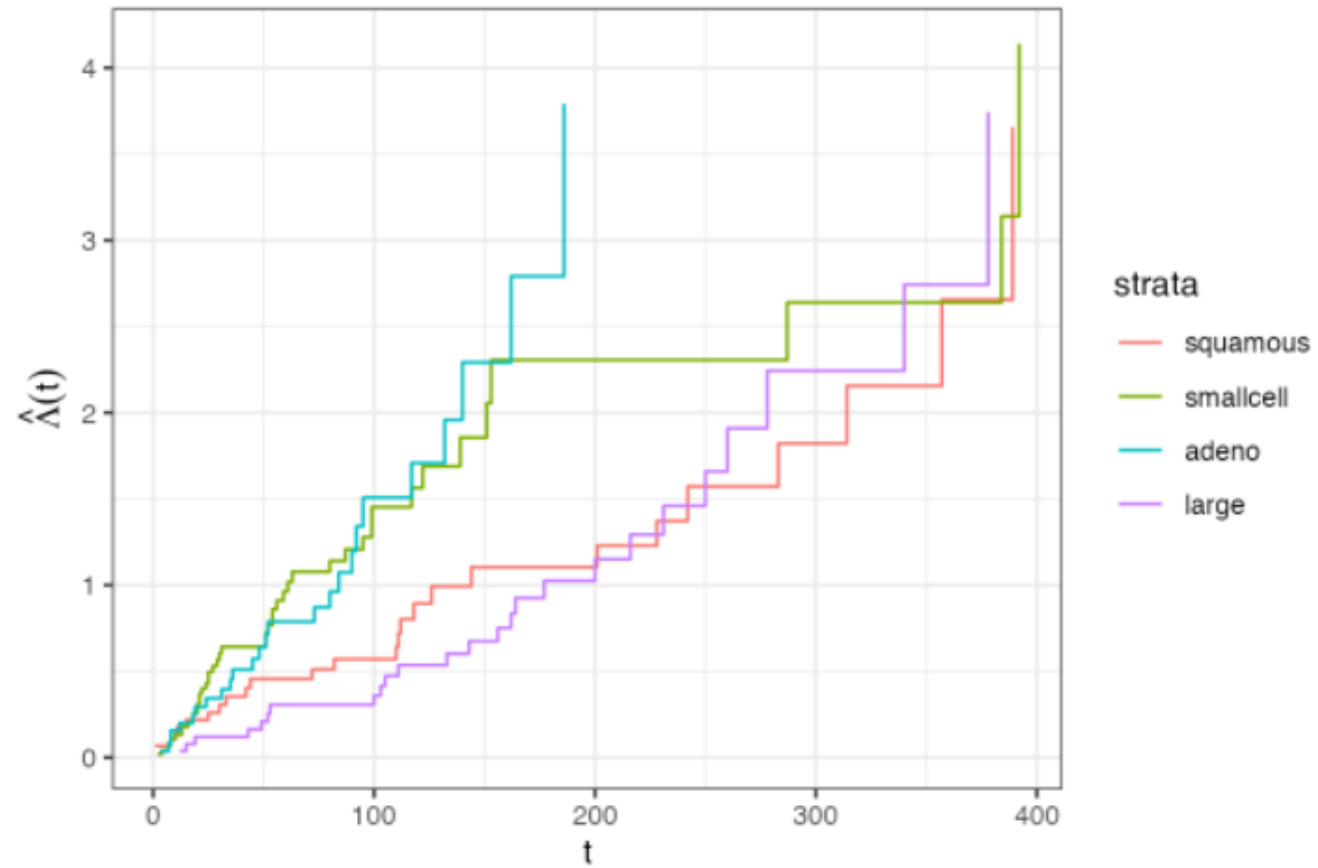
- Roimi, Michael, et al. "Development and validation of a machine learning model predicting illness trajectory and hospital utilization of COVID-19 patients: A nationwide study." *Journal of the American Medical Informatics Association* 28.6 (2021): 1188-1196.



Extension 4 – Strata

- We can relax the proportional hazards assumption by allowing a different baseline hazard per strata of some variable
- We would usually only use this option for very important categorical variables
- Obviously, we wouldn't get a hazard ratio for that variable, so less interpretability
- Also, more risk for overfitting

Extension 4 – Strata (2)



Extension 5 – Time Varying Coefficients

- Another way to relax the proportional hazards assumption, this time in a way that can also work with continuous variables, is to define a time-dependent function for the effect
- This is different than time varying variables, that assume a constant effect but a different covariate value:

$$\lambda(t) = \lambda_0(t)e^{\beta X(t)}$$

$$\lambda(t) = \lambda_0(t)e^{\beta(t)X}$$

Extension 5 – Time Varying Coefficients (2)

- This would look something like:

```
vfit3 <- coxph(Surv(time, status) ~ trt + prior + karno + tt(karno),  
              data=veteran,  
              tt = function(x, t, ...) x * log(t+20))
```

- In practice it is not widely used, as it requires the data scientist to pre-guess the correct function

Modern Survival Analysis:

Discrete-Time Hazard Models

- Both KM and Cox teach us that once you have a model for the hazards, you get everything else “for free” (survival curve, censoring etc.)
- So, thinking of this basic model $\text{logit}(h_t(x)) = \alpha_t + f(x, t)$, we could model f any way we want:
 - logistic regression (=Pooled Logistic Regression, PLR)
 - gradient boosted trees
 - neural networks

Modern Survival Analysis:

Random Survival Forests

- Ishwaran (2008)
- Splitting uses survival-aware criteria – e.g., the log-rank test statistic
- Stopping rules demand a minimum of events
- At the terminal nodes, we estimate using, e.g., the Kaplan-Meier estimator and average
- Many (many...) hyper-parameters and extensions to handle things like competing risks, variable importance, uncertainty, etc.

Modern Survival Analysis:

Boosted Survival Trees

- Boosting with a survival-oriented loss (such as Cox's partial likelihood)
- Well-supported in software
 - scikit-learn: *GradientBoostingSurvivalAnalysis*
 - Xgboost: *survival:cox* & *survival:aft*
- A strong and underutilized tool for survival analysis

Picking a Model

- What happens if we have follow-up-time-related issues in our data such as different follow-up periods and censoring, and we try to use a “flat” model?
- We have to make a choice
 - we could include these people, pretending their follow-up time was similar, incurring misclassification error
 - we could not include them, reducing the sample and resulting in selection bias if drop-out is related to the outcome
- So, you would think that choosing between “flat” models (logistic regression, xgboost, etc.) and time-aware models is easy
 - If we need to model time, we have to use a time-aware model!
 - But flat models, on average, have a richer software ecosystem and tend to give better performance, so the choice is actually quite hard